Integración de Set, Map, Enumeraciones y Excepciones en el Proyecto RPG

Taller Práctico:

1. Contexto y Objetivos del Taller

1.1. Contexto del Proyecto Integrador

En las unidades previas del curso se ha desarrollado la base estructural del **Simulador de Batalla RPG / Arena de Héroes**, modelando entidades básicas mediante programación orientada a objetos (POO) e implementando lógica condicional y estructuras lineales simples. Sin embargo, el diseño actual padece de acoplamiento débil hacia el dominio debido al uso extenso de primitivos y cadenas de texto libres (`String`) para definir estados, roles y atributos.

En este taller de arquitectura de software, se abordará la refactorización profunda del núcleo del sistema. Se sustituirán los atributos basados en texto plano por **tipos fuertemente tipados (`**enum**`)**, se integrará el marco de colecciones de Java (**Java Collections Framework**) mediante el uso de `Set` para garantizar la unicidad de elementos y `Map` para búsquedas dinámicas y contadores, y se dotará al sistema de un mecanismo robusto de control de errores mediante **excepciones personalizadas de dominio** y estructuras de control `try-catch-finally` .

1.2. Objetivos de Aprendizaje

Al finalizar este taller práctico, el estudiante estará capacitado para:

- **Diseñar e implementar tipos enumerados (`**enum**`) con estado interno y comportamiento**, sustituyendo constantes místicas (*magic numbers/strings*) y mejorando la seguridad en tiempo de compilación.
- **Garantizar la unicidad y el ordenamiento de conjuntos de datos** en el modelo de dominio mediante el uso estratégico de las interfaces y clases del marco de colecciones: `HashSet` para elementos sin duplicados y `TreeSet` para elementos ordenados naturalmente.
- **Modelar repositorios y contadores dinámicos tipo clave-valor** utilizando `HashMap` para accesos O(1) y `TreeMap` para navegación ordenada de estadísticas e inventarios.
- **Construir una jerarquía sólida de excepciones de dominio** (tanto verificadas como no verificadas) para encapsular las fallas de lógica de negocio en el combate.
- **Implementar patrones de captura y propagación de errores (`**try-catch-finally**`)** en la interfaz de consola, procesando entradas de usuario con `Scanner` de forma resiliente ante fallas de formato (`InputMismatchException`).
- **Utilizar herramientas avanzadas de depuración en IntelliJ IDEA**, analizando el estado de colecciones y el rastreo de pila (*stack trace*) durante la ejecución.

2. Sección 1: Rediseño con Enumeraciones (`enum`)

La refactorización del modelo de dominio exige eliminar el uso de cadenas arbitrarias para definir la clase, estado y rareza de las entidades. El estudiante debe implementar tres tipos enumerados bien estructurados en paquetes correspondientes al modelo (`model.enums`).

2.1. Especificación de `TipoPersonaje`

El tipo enumerado `TipoPersonaje` definirá los arquetipos de héroes disponibles en el juego. No debe limitarse a ser un mero identificador, sino que debe incorporar atributos de estado interno inmutables que afecten las estadísticas base del personaje:

- **Constantes:** `GUERRERO` , `MAGO` , `ARQUERO` .
- **Atributos Internos (`**private final**`):**
  - `bonificadorAtaqueBase` (`double`): Incremento porcentual o valor fijo al ataque.
  - `bonificadorDefensaBase` (`double`): Incremento al valor de mitigación de daño.
- **Constructores y Métodos:** Constructor privado para inicializar los atributos de cada constante y métodos *getter* públicos para la lectura de los bonificadores.

2.2. Especificación de `EstadoPersonaje`

Para gestionar el ciclo de vida y las alteraciones de estado de los héroes en la arena de combate, se requiere la especificación del enum `EstadoPersonaje` :

- **Constantes:** `VIVO` , `DERROTADO` , `ENVENENADO` , `ATURDIDO` .
- **Comportamiento Esperado:** Servirá para la toma de decisiones en el bucle de combate. Por ejemplo, un personaje en estado `ATURDIDO` pierde su turno de ataque, mientras que un estado `ENVENENADO` aplicará daño por turno durante la fase de actualización.

2.3. Especificación de `RaridadItem`

Los ítems del equipamiento e inventario deben clasificarse jerárquicamente para calcular su valor comercial o su impacto en estadísticas mediante `RaridadItem` :

- **Constantes:** `COMUN` , `RARO` , `LEGENDARIO` .
- **Atributos Internos:**
  - `multiplicadorStat` (`double`): Factor numérico (ej. 1.0 para Común, 1.5 para Raro, 2.5 para Legendario) utilizado para escalar los atributos de los ítems.
- **Encapsulamiento:** Atributo final con su correspondiente método *getter* `getMultiplicadorStat()` .

2.4. Guía de Refactorización

El estudiante deberá realizar las siguientes modificaciones en las clases de entidad existentes (ej. `Personaje` , `Item`):

1. Reemplazar la variable `String tipo` en la clase `Personaje` por la propiedad `private TipoPersonaje tipoPersonaje` .

2. Sustituir las banderas booleanas de vida/muerte o cadenas sueltas por la propiedad `private EstadoPersonaje estado`.
3. Actualizar los constructores de `Personaje` para exigir el tipo `TipoPersonaje` y la inicialización por defecto del estado en `EstadoPersonaje.VIVO`.
4. Modificar los métodos de cálculo de atributos para que consuman directamente los bonificadores encapsulados dentro del enum asignado al personaje.

3. Sección 2: Gestión de Logros y Habilidades con `Set`

Para garantizar la integridad semántica de los datos, ciertas colecciones del dominio no deben permitir elementos duplicados bajo ninguna circunstancia. Se utilizará la interfaz `java.util.Set` para resolver dos casos de uso de la arena.

3.1. Libro de Hechizos / Habilidades Únicas (`HashSet`)

Cada personaje dispondrá de un libro de habilidades o hechizos aprendidos. Dado que un personaje no puede aprender la misma habilidad dos veces y el orden de aprendizaje no altera la validez del libro, se requiere el uso de `HashSet<String>`.

- **Regla de Negocio:** La inserción de una habilidad debe verificar si el nombre ya existe. Si la habilidad ya está presente, la operación no debe alterar la colección y debe informar al sistema el rechazo del duplicado.

3.2. Registro de Logros Desbloqueados (`TreeSet`)

El perfil del jugador mantendrá un registro global de logros desbloqueados durante sus batallas (ej. "Primera Victoria", "Maestro del Mana", "Cazador de Dragones").

- **Regla de Negocio:** Este registro debe garantizar la unicidad de los logros y presentarlos automáticamente **ordenados alfabéticamente** para su despliegue en la interfaz gráfica/consola. Para cumplir este requerimiento, se especifica la implementación mediante `TreeSet<String>`.

3.3. Requerimientos de Métodos

El estudiante debe integrar en las clases del modelo (`Personaje` y `PerfilJugador`) los métodos detallados en la siguiente especificación funcional:

Nombre del Método	Firma	Tipo de Set	Comportamiento Esperado
`aprenderHabilidad`	`public boolean aprenderHabilidad(String habilidad)`	`HashSet<String>`	Intenta agregar la habilidad al conjunto. Retorna `true` si fue agregada con éxito; retorna

			`false` si la habilidad ya existía (rechazo de duplicado).
`obtenerHabilidades`	`public Set<String> obtenerHabilidades()`	`HashSet<String>`	Devuelve una vista no modificable o la referencia del conjunto de habilidades aprendidas.
`registrarLogro`	`public boolean registrarLogro(String logro)`	`TreeSet<String>`	Agrega un nuevo logro al registro ordenado. Retorna `true` si es un logro nuevo y `false` si ya había sido desbloqueado.
`listarLogrosOrdenados`	`public Set<String> listarLogrosOrdenados()`	`TreeSet<String>`	Retorna el conjunto de logros. Se debe verificar que la iteración sobre este conjunto siga estrictamente el orden alfabético natural.

4. Sección 3: Bestiario y Estadísticas de Juego con `Map`

El motor del juego requiere estructuras de tipo clave-valor para gestionar diccionarios de datos, contadores dinámicos e inventarios de tiendas.

4.1. Contador de Enemigos Derrotados / Bestiario (`HashMap`)

Se debe diseñar la clase `Bestiario` dentro del paquete `engine`. Esta clase contendrá un mapa interno que relacionará el tipo o nombre del enemigo con la cantidad total de veces que ha sido derrotado por el jugador:

- **Estructura:** `Map<String, Integer>` instanciado como `HashMap<String, Integer>`.
- **Propósito:** Ofrecer lecturas y escrituras de alta velocidad (O(1)) al registrar las bajas en combate.

4.2. Diccionario de Estadísticas / Inventario Tienda (`TreeMap`)

Para el catálogo de la tienda de ítems o el reporte consolidado de estadísticas globales de combate, se requiere ordenar las entradas de manera alfabética en base a sus claves:

- **Estructura:** `Map<String, Double>` instanciado como `TreeMap<String, Double>`.
- **Propósito:** Mantener un listado de precios de ítems o promedios de daño ordenado automáticamente por el nombre del objeto o parámetro estadístico.

4.3. Especificaciones de Operación de Mapas

El estudiante debe implementar las siguientes operaciones en el motor del juego utilizando los métodos nativos de la interfaz `Map`:

1. **Verificación de Claves (`**containsKey**`):** Antes de consultar o incrementar una estadística, el sistema debe comprobar si la clave ya está registrada en el mapa.
2. **Lectura Segura (`**getOrDefault**`):** Al incrementar el contador de un enemigo en el bestiario, se debe utilizar `mapa.getOrDefault(nombreEnemigo, 0)` para evitar excepciones por punteros nulos (`NullPointerException`) en enemigos no registrados previamente.
3. **Actualización de Valores:** Actualizar la clave incrementando su valor en 1 tras cada victoria en la arena.
4. **Recorrido de Entradas (`**entrySet**`):** Para presentar reportes en consola, se exige iterar sobre las entradas del mapa utilizando `for (Map.Entry<K, V> entry : mapa.entrySet())`, mostrando la clave y el valor formateados.

5. Sección 4: Manejo de Excepciones Personalizadas y Robustez

Para evitar que la aplicación colapse ante estados inválidos de juego o entradas erróneas en la consola, se diseñará un sistema de excepciones robusto enfocado en la lógica de negocio.

5.1. Jerarquía de Excepciones de Dominio

El estudiante debe crear un paquete `exception` y estructurar las siguientes clases de excepción personalizadas:

- `**JuegoException**`: Clase base abstracta que extiende de `Exception` (excepción verificada). Todas las excepciones del dominio deben heredar de esta clase.
- `**SinManaException**`: Excepción verificada (`extends JuegoException`). Disparada cuando un personaje intenta ejecutar una habilidad mágica sin el nivel de maná requerido.
- `**VidaInsuficienteException**`: Excepción no verificada (`extends RuntimeException`). Disparada cuando se intenta realizar una acción de costo sobre un personaje que sobrepasa sus puntos de vida actuales.
- `**CapacidadExcedidaException**`: Excepción verificada (`extends JuegoException`). Disparada al intentar agregar un ítem o habilidad cuando el contenedor correspondiente ha alcanzado su límite máximo.

5.2. Reglas de Disparo (Throwing Conditions)

El motor de combate y las entidades deben evaluar condiciones límite y arrojar (*throw*) explícitamente las excepciones según la siguiente tabla de reglas:

Acción del Juego	Condición de Disparo	Excepción a Lanza	Tipo de Excepción
Lanzar Hechizo	`costoMana > manaActual`	`SinManaException`	Verificada (`Checked`)
Consumir Puntos de Vida	`costoVida >= vidaActual`	`VidaInsuficienteException`	No verificada (`Unchecked`)
Inventario / Libro Lleno	`totalElementos >= capacidadMaxima`	`CapacidadExcedidaException`	Verificada (`Checked`)

5.3. Control de Errores en Entrada de Usuario (`try-catch-finally`)

La interacción por consola mediada por la clase `Scanner` debe ser blindada contra datos de entrada incompatibles.

- **Especificación:** Cuando el usuario deba ingresar un valor numérico (por ejemplo, al seleccionar una opción del menú o asignar puntos de atributo), el código de lectura debe estar encapsulado dentro de un bloque `try-catch`.
- **Captura:** Se debe capturar explícitamente la excepción `java.util.InputMismatchException`.
- **Tratamiento:** En caso de error, el programa no debe finalizar abruptamente; debe limpiar el búfer del `Scanner` (`scanner.nextLine()`), imprimir un mensaje orientativo en la consola de error (`System.err`) y permitir reintentar la operación.
- **Bloque `**finally**` :** Utilizar el bloque `finally` para asegurar la ejecución de tareas de limpieza o confirmación de ciclo de lectura cuando corresponda.

6. Sección 5: Reto Integrador (Evolución de la Arena RPG)

6.1. Requisitos de Menú Interactivo

El estudiante debe reestructurar el punto de entrada de la aplicación (`Main.java`) implementando un bucle de control interactivo que presente las siguientes cinco opciones:

1. **Crear Personaje:** Solicitar nombre y seleccionar el `TipoPersonaje` utilizando los valores del enum. Inicializar sus colecciones internas de habilidades y su estado en `EstadoPersonaje.VIVO`.
2. **Asignar Habilidad Única:** Solicitar el nombre de una habilidad e intentar registrarla en el `HashSet` del personaje. Si el método retorna `false`, mostrar un mensaje advirtiendo que la habilidad ya está aprendida.
3. **Registrar Victoria en Bestiario:** Solicitar el nombre del enemigo derrotado e incrementar su contador en el `HashMap` del bestiario mediante `getOrDefault`. Si es la primera victoria de este tipo, registrar automáticamente un logro en el `TreeSet`.
4. **Ejecutar Ataque Especial:** Solicitar el costo de maná del ataque. Llamar al método de lanzamiento del personaje. Manejar la posible excepción `SinManaException` mediante un bloque `try-catch`.

5. **Consultar Bestiario y Logros:** Recorrer y desplegar en pantalla el mapa del bestiario (`entrySet`) y el conjunto de logros ordenados (`TreeSet`).

6.2. Especificación de Arrojo y Captura de Flujo

El flujo de ejecución debe seguir estrictamente el patrón de propagación de excepciones en arquitectura por capas:

1. **Capa de Dominio (`**Personaje**` / `**CombateEngine**`):** Evalúa la regla de negocio. Si falla, instancia y lanza la excepción correspondiente (`throw new SinManaException(...)`). Las excepciones verificadas deben declararse en la firma del método (`throws SinManaException`).
2. **Capa de Interfaz / Control (`**Main**`):** Captura las excepciones de dominio arrojadas por la capa inferior utilizando bloques `catch` específicos.
3. **Mensajería:** El bloque `catch` debe extraer el mensaje informativo de la excepción (`e.getMessage()`) y desplegarlo al usuario sin interrumpir el bucle principal del menú de juego.

7. Sección 6: Puntos Críticos de Depuración en IntelliJ IDEA

Para garantizar la calidad del software desarrollado, el estudiante debe auditar su código previniendo antipatrones comunes y utilizando la suite de depuración de IntelliJ IDEA.

7.1. Trampas de Código y Antipatrones

El estudiante debe revisar y corregir activamente los siguientes errores de programación en sus entregables:

- **Ocultamiento de Excepciones (*Exception Swallowing*):** Evitar bloques `catch` vacíos o que solo impriman `e.printStackTrace()` sin restaurar el flujo o notificar al usuario.
- **Pérdida de la Causa Raíz:** Al reenvolver excepciones, se debe pasar la excepción original como causa (`super(mensaje, causa)`) para no perder el rastro de la pila.
- **Acceso a Claves Nulas o Inexistentes:** Intentar operaciones sobre referencias `null` dentro de un `Map` o asumir que `map.get(clave)` nunca retornará `null` .
- **Violación del Contrato `**equals()**` y `**hashCode()**` :** Si se evoluciona el `Set` o `Map` para utilizar objetos de clases personalizadas (ej. `Set<Habilidad>`) en lugar de `String` , es un requisito estricto sobrescribir correctamente los métodos `equals(Object o)` y `hashCode()` . De lo contrario, las colecciones basadas en *hash* (`HashSet` , `HashMap`) fallarán al detectar duplicados, insertando objetos conceptualmente idénticos.

7.2. Lista de Comprobación de Depuración (Debugging Checklist)

El estudiante debe ejecutar la siguiente guía de comprobación paso a paso dentro del IDE IntelliJ IDEA para verificar el correcto funcionamiento del sistema:

1. **Configuración de Puntos de Interrupción (*Breakpoints*):**
   - Establecer un *Breakpoint* en la línea donde se invoca el método `aprenderHabilidad()` .

- Establecer un *Breakpoint* condicional en la línea de lanzamiento de `SinManaException` (Configurar condición: `manaActual < costo`).

2. **Inspección de Colecciones en Tiempo de Ejecución:**
   - Iniciar la aplicación en modo Debug (`Shift + F9`).
   - Navegar hasta la pestaña **Variables** en el panel inferior de IntelliJ IDEA.
   - Desplegar la vista de objeto del personaje y examinar la estructura interna del `HashSet` de habilidades y el `HashMap` del bestiario, verificando la tabla de *buckets* y el tamaño (`size`).
3. **Evaluación de Expresiones al Vuelo (*Evaluate Expression*):**
   - Mientras la ejecución esté pausada en un punto de interrupción dentro de la fase de ataque, presionar `Alt + F8` para abrir la herramienta **Evaluate Expression**.
   - Probar manualmente consultas sobre las colecciones, tales como: `bestiarioMap.containsKey("Orcos")` o `personaje.getHabilidades().contains("Bola de Fuego")`.
   - Validar que los resultados booleanos e integrales devueltos por la ventana de evaluación coincidan con la lógica esperada antes de reanudar la ejecución del programa (`F9`).